

Lista de Exercícios — Árvores Binárias de Busca (ABB)

Objetivo

Nesta lista, iremos trabalhar com operações fundamentais em Árvores Binárias de Busca (ABB), utilizando a mesma lógica implementada em sala de aula.

A representação da árvore será feita utilizando dicionários em Python.

Exercício 1 — Construção da Árvore

Considere a lista abaixo:

```
valores = [53, 30, 72, 14, 39, 61, 84, 9, 23, 34, 49, 79]
```

Implemente o algoritmo de inserção em uma árvore binária de busca utilizando dicionários.

Cada nó deve possuir:

```
{"esquerda": None, "direita": None}
```

Ao final:

1. Mostre o dicionário completo da árvore.
 2. Informe qual é a raiz.
 3. Explique por que cada valor foi para esquerda ou direita.
-

Exercício 2 — Inserção de Novos Valores

Utilizando a árvore criada no exercício anterior:

1. Insira os seguintes valores:

```
[65, 90, 18]
```

1. Mostre como ficou a árvore após cada inserção.
2. Indique o caminho percorrido até inserir cada valor.

Exemplo esperado:

65 53 72 61

Exercício 3 — Busca de Elementos

Implemente um algoritmo que procure um valor dentro da árvore.

Seu algoritmo deve:

1. Informar se o valor foi encontrado.
2. Mostrar o caminho percorrido.
3. Contar quantos nós foram visitados.

Teste para os seguintes valores:

49
84
100

Exercício 4 — Travessia Pré-Ordem

Implemente a travessia em pré-ordem utilizando pilha.

Relembrando:

- Visita a raiz
- Percorre esquerda
- Percorre direita

Ao final:

1. Mostre a lista resultante.
2. Explique por que a pilha funciona nesse algoritmo.

Exercício 5 — Travessia Em Ordem

Implemente a travessia em ordem utilizando pilha.

Ao final:

1. Mostre a lista resultante.
2. Explique por que o resultado aparece ordenado.
3. Explique a diferença entre pilha e recursão nesse caso.

Exercício 6 — Travessia Pós-Ordem

Implemente a travessia pós-ordem utilizando duas pilhas.

Ao final:

1. Mostre o resultado final.
 2. Explique por que são necessárias duas pilhas.
 3. Compare com pré-ordem e em-ordem.
-

Exercício 7 — Altura da Árvore

Implemente um algoritmo que calcule a altura da árvore.

Definição:

- A altura é o maior número de níveis da árvore.

Seu programa deve:

1. Informar a altura da árvore.
 2. Mostrar qual caminho gera essa altura.
-

Exercício 8 — Contagem de Nós

Crie algoritmos para:

1. Contar quantos nós existem na árvore.
 2. Contar quantos nós folha existem.
 3. Contar quantos nós possuem dois filhos.
 4. Contar quantos nós possuem apenas um filho.
-

Exercício 9 — Menor e Maior Valor

Implemente algoritmos que encontrem:

1. O menor valor da árvore.
2. O maior valor da árvore.

Além disso:

- Mostre o caminho percorrido.

- Explique por que esses caminhos funcionam.
-

Exercício 10 — Remoção de Nó Folha

Implemente a remoção do valor:

9

Seu algoritmo deve:

1. Identificar o pai do nó.
 2. Remover corretamente o elemento.
 3. Mostrar a árvore antes e depois.
-

Exercício 11 — Remoção de Nó com Um Filho

Implemente a remoção do valor:

84

Considere que ele possui apenas um filho.

Seu algoritmo deve:

1. Ajustar corretamente os ponteiros.
 2. Mostrar o que mudou na árvore.
 3. Explicar a lógica da substituição.
-

Exercício 12 — Remoção de Nó com Dois Filhos

Implemente a remoção do valor:

72

Seu algoritmo deve:

1. Encontrar o sucessor.
2. Mostrar qual foi o sucessor encontrado.
3. Explicar por que ele foi escolhido.
4. Mostrar a árvore final.

Exercício 13 — Busca de Profundidade

Implemente uma busca em profundidade (DFS) utilizando pilha.

Seu algoritmo deve:

1. Mostrar a ordem dos nós visitados.
 2. Procurar um valor específico.
 3. Informar em qual nível o valor foi encontrado.
-

Exercício 14 — Busca em Largura

Implemente uma busca em largura (BFS) utilizando fila.

Seu programa deve:

1. Mostrar os nós nível por nível.
 2. Informar quantos níveis existem.
 3. Comparar BFS com DFS.
-

Exercício 15 — Verificação de Existência

Crie uma função:

```
existe(arvore, valor)
```

A função deve retornar:

- True → se o valor existir
- False → caso contrário

Teste para:

```
23  
100  
61
```

Exercício 16 — Soma dos Valores da Árvore

Crie um algoritmo que:

1. Percorra toda a árvore.
 2. Some todos os valores armazenados.
 3. Mostre a média dos elementos.
-

Exercício 17 — Conversão para Lista Ordenada

Utilize a travessia em ordem para:

1. Converter a árvore em uma lista ordenada.
 2. Mostrar o resultado.
 3. Explicar por que a ABB gera uma lista ordenada.
-

Exercício 18 — Verificação de Balanceamento

Implemente um algoritmo que verifique se a árvore está balanceada.

Considere:

- Uma árvore balanceada possui alturas semelhantes entre subárvores.

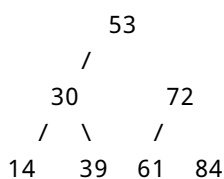
Seu programa deve:

1. Calcular alturas.
 2. Comparar os lados.
 3. Informar se a árvore está balanceada.
-

Exercício 19 — Desenho da Árvore

Represente visualmente a árvore.

Exemplo:



Depois:

1. Faça o desenho após inserções.
 2. Faça o desenho após remoções.
-

Exercício 20 — Desafio

Crie um menu interativo com as opções:

- 1 - Inserir valor
- 2 - Buscar valor
- 3 - Remover valor
- 4 - Mostrar pré-ordem
- 5 - Mostrar em ordem
- 6 - Mostrar pós-ordem
- 7 - Mostrar árvore
- 8 - Encerrar

O sistema deve funcionar continuamente até o usuário escolher sair.

Exercícios Teóricos

Exercício 21

Explique a diferença entre:

- árvore binária
 - árvore binária de busca
-

Exercício 22

Explique:

1. O que é um nó raiz.
 2. O que é um nó folha.
 3. O que é altura.
 4. O que é profundidade.
 5. O que é subárvore.
-

Exercício 23

Qual é a complexidade média da busca em uma ABB?

Explique:

- Melhor caso
 - Caso médio
 - Pior caso
-

Exercício 24

Explique por que árvores degeneradas podem se tornar semelhantes a listas encadeadas.

Dê exemplos.

Desafio Extra

Pesquise e explique:

1. Árvores AVL
 2. Árvores Rubro-Negras
 3. Diferença entre ABB comum e AVL
 4. Por que árvores balanceadas são importantes
-

Observações

- Todos os exercícios devem ser implementados em Python.
- Evite utilizar bibliotecas prontas.
- Priorize a lógica manual.
- Utilize comentários explicando o algoritmo.
- Sempre teste com diferentes valores.